

VIP Cheatsheet:

Transformers e Grandes Modelos de Linguagem

Afshine AMIDI e Shervine AMIDI
Traduzido por Felipe Galiza

24 de maio de 2026

Esta VIP Cheatsheet apresenta uma visão geral do conteúdo do livro “Super Study Guide: Transformers & Large Language Models”, que contém aproximadamente 600 ilustrações distribuídas em 250 páginas, abordando em profundidade os conceitos relacionados ao tema. Mais detalhes disponíveis em <https://superstudy.guide>.

1 Fundamentos

1.1 Tokens

Definição – Um *token* é uma unidade indivisível de texto, como uma palavra, subpalavra ou caractere, e faz parte de um vocabulário predefinido.

Observação: O token desconhecido [UNK] representa partes de texto não reconhecidas, enquanto o token de preenchimento [PAD] é usado para completar posições vazias e manter o comprimento consistente das sequências de entrada.

Tokenizador – Um *tokenizador* T divide o texto em tokens de um nível arbitrário de granularidade.

esse ursinho de pelúcia é muuuito fofo → T → [esse] [ursinho de pelúcia] [é] [UNK] [fofo] [PAD]... [PAD]

Principais tipos de tokenização:

Tipo	Prós	Contras	Ilustração
Palavra	- Fácil de interpretar - Sequência curta	- Vocabulário muito grande - Não lida bem com variações de palavras	ursinho de pelúcia
Subpalavra	- Aproveita raízes das palavras - Gera embeddings intuitivos	- Sequência mais longa - Tokenização mais complexa	urs:##inho de: pel:##úcia
Caractere Byte	- Sem problemas de vocabulário desconhecido - Vocabulário pequeno	- Sequência muito longa - Padrões difíceis de interpretar por serem muito de baixo nível	u r s i n h o d e p e l ú c i a

Observação: Byte-Pair Encoding (BPE) e Unigramas são tokenizadores de nível subpalavra amplamente utilizados.

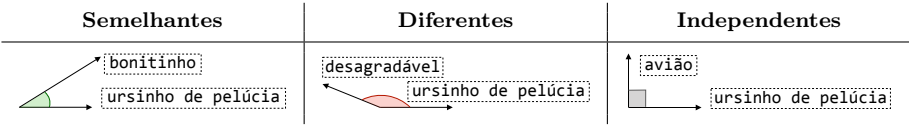
1.2 Embeddings

Definição – Um *embedding* é uma representação numérica de um elemento (por exemplo, token ou sentença), caracterizada por um vetor $x \in \mathbb{R}^n$.

Similaridade – A *similaridade de cosseno* entre dois tokens t_1 e t_2 é dada por:

similaridade(t_1, t_2) = $\frac{t_1 \cdot t_2}{||t_1|| ||t_2||} = \cos(\theta) \in [-1, 1]$

O ângulo θ caracteriza a similaridade entre os dois tokens:

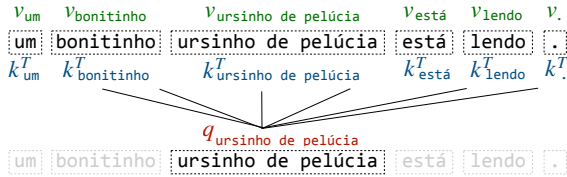


Observação: Vizinhos aproximados mais próximos (Approximate Nearest Neighbors, ANN) e Hashing sensível à localidade (Locality Sensitive Hashing, LSH) são métodos que permitem aproximar a operação de similaridade de forma eficiente em grandes bases de dados.

2 Transformers

2.1 Mecanismo de Atenção

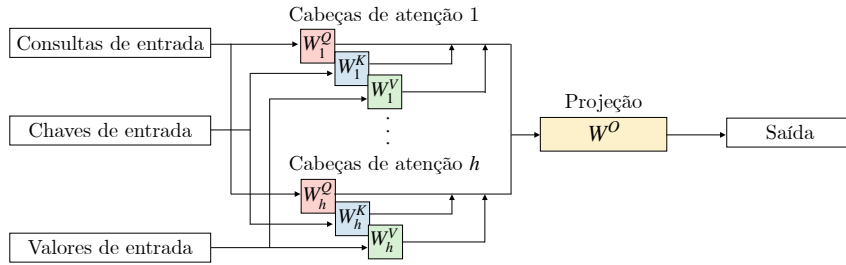
Fórmula – Dada uma *consulta* q , queremos saber para qual *chave* k ela deve prestar “atenção” em relação ao *valor* associado v .



A atenção pode ser calculada de forma eficiente usando matrizes Q, K, V , que contêm consultas q , chaves k e valores v , respectivamente, junto com a dimensão d_k das chaves:

atenção = $\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$

□ **MHA** – Uma camada de *Atenção Multi-Cabeça* (**Multi-Head Attention**, MHA) realiza cálculos de atenção em múltiplas cabeças e projeta o resultado no espaço de saída.

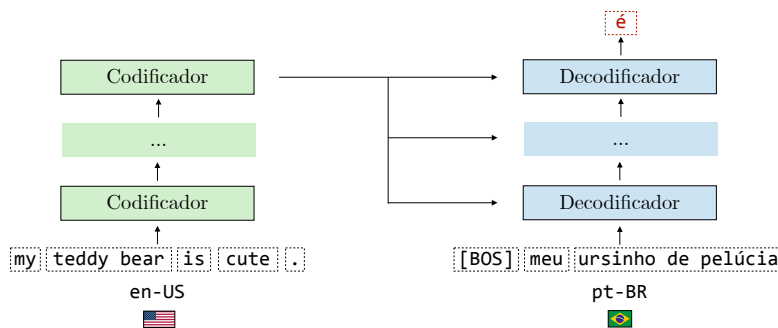


Ela é composta por h cabeças de atenção, bem como pelas matrizes W^Q, W^K, W^V , que projetam a entrada para obter consultas Q , chaves K e valores V . A projeção é realizada usando a matriz W^O .

Observação: O Mecanismo de Atenção de Consultas Agrupadas (Grouped-Query Attention, GQA) e a Atenção de Múltiplas Consultas (Multi-Query Attention, MQA) são variações do MHA que reduzem o custo computacional ao compartilhar chaves e valores entre as cabeças de atenção.

2.2 Arquitetura

□ **Visão Geral** – O *Transformer* é um modelo histórico baseado em autoatenção, composto por codificadores (encoders) e decodificadores (decoders). Os encoders produzem embeddings representativos da entrada, que são usados pelos decoders para prever o próximo token da sequência.

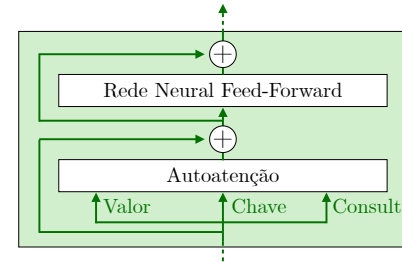


Observação: Embora inicialmente proposto para tarefas de tradução, o Transformer é hoje amplamente utilizado em diversas aplicações.

□ **Componentes** – O *encoder* e o *decoder* são dois componentes fundamentais do Transformer e possuem papéis distintos:

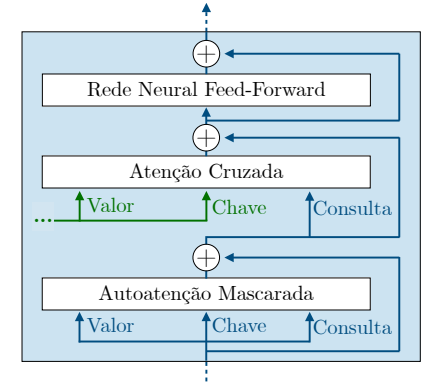
Codificador

Embeddings codificados capturam o significado da entrada



Decodificador

Embeddings decodificados capturam o significado tanto da entrada quanto da saída gerada até o momento

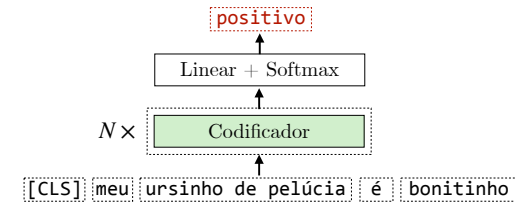


□ **Embeddings de posição** – *Embeddings* de posição informam a posição do token na frase e têm a mesma dimensão dos embeddings de token. Eles podem ser definidos arbitrariamente ou aprendidos a partir dos dados.

Observação: Embeddings de Posição Rotativos (Rotary Position Embeddings, RoPE) são uma variação eficiente que rotaciona os vetores de consulta e chave para incorporar informações de posição relativa.

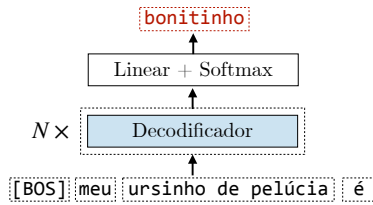
2.3 Variantes

□ **Apenas codificador** – BERT (**Bidirectional Encoder Representations from Transformers**) é um modelo baseado em Transformers composto por camadas de codificadores que recebem texto como entrada e geram embeddings que posteriormente podem ser utilizados em tarefas como classificação.



O token [CLS] é adicionado no início da sequência para capturar o significado da sentença. Seu embedding codificado é geralmente utilizado em tarefas posteriores, como análise de sentimento.

□ **Apenas decodificador** – GPT (**Generative Pre-trained Transformer**) é um modelo autor-regressivo baseado em Transformers composto por decodificadores empilhados. Diferente do BERT e de seus derivados, o GPT trata todos os problemas como problemas de texto para texto.



A maioria dos grandes modelos de linguagem do estado da arte atual é baseada em arquiteturas de apenas decodificador, como as variações do GPT, LLaMA, Mistral, Gemma, DeepSeek, etc.

Observação: Modelos codificador-decodificador, como o T5, também são autorregressivos e compartilham muitas características com modelos de arquitetura apenas decodificadora.

2.4 Otimizações

□ **Aproximações do Mecanismo de Atenção** – Os cálculos do mecanismo de atenção possuem complexidade $\mathcal{O}(n^2)$, o que pode ser caro à medida que o tamanho n da sequência aumenta. Existem dois métodos principais de aproximação:

- *Esparsidade*: A autoatenção não acontece em toda a sequência, mas apenas entre os tokens mais relevantes.



- *Posto baixo*: A fórmula do mecanismo de atenção é simplificada como o produto de matrizes de posto baixo, reduzindo a carga computacional.

□ **Atenção Flash** – *Atenção flash* é um método exato que otimiza os cálculos da atenção aproveitando de maneira inteligente o hardware da GPU, utilizando a memória estática de acesso aleatório (*Static Random-Access Memory*, SRAM) para operações matriciais antes de escrever os resultados na memória de alta largura de banda (*High Bandwidth Memory*, HBM), que é mais lenta.

Observação: Na prática, isso reduz o uso da memória e acelera os cálculos.

3 Grandes Modelos de Linguagem

3.1 Visão Geral

□ **Definição** – Um *Grande Modelo de Linguagem* (*Large Language Model*, LLM) é um modelo baseado em Transformer com potentes capacidades de processamento de linguagem natural. É “grande” no sentido de que tipicamente possui bilhões de parâmetros.

□ **Ciclo de vida** – Um LLM é treinado em 3 passos: pré-treino, ajuste fino e ajuste de preferências.

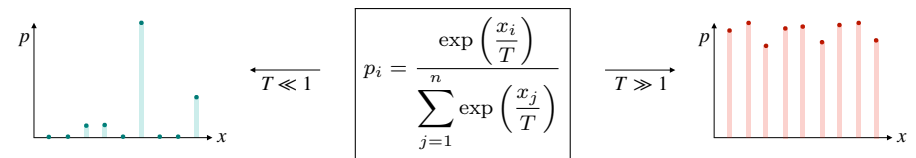


Ajuste fino e ajuste de preferências são abordagens pós-treinamento que visam alinhar o modelo para realizar tarefas específicas.

3.2 Prompting

□ **Tamanho do contexto** – O tamanho do contexto de um modelo é o número máximo de tokens que podem ser incluídos na entrada. Normalmente, varia de dezenas de milhares até milhões de tokens.

□ **Amostragem de decodificação** – As predições de tokens são amostradas a partir da distribuição de probabilidade predita p_i , que por sua vez é controlada pelo hiperparâmetro de temperatura T .



Observação: Temperaturas altas geram saídas mais criativas, enquanto temperaturas baixas geram saídas mais determinísticas.

□ **Cadeia de Pensamento** – A Cadeia de Pensamento (*Chain-of-Thought*, CoT) é um processo de raciocínio em que o modelo decompõe problemas complexos em uma série de passos intermediários. Isso ajuda o modelo a gerar a resposta final correta. A Árvore de Pensamentos (*Tree of Thoughts*, ToT) é uma versão mais avançada do CoT.

Observação: A autoconsistência é um método que agrega respostas ao longo dos caminhos de raciocínio do CoT.

3.3 Ajuste fino

□ **SFT** – O ajuste fino supervisionado (*Supervised FineTuning*, SFT) é uma abordagem de pós-treinamento que alinha o comportamento do modelo com a tarefa final. Baseia-se em pares de entrada-saída de alta qualidade alinhados com a tarefa.

Observação: Se os dados de SFT forem sobre instruções, então esse passo é chamado de “ajuste de instruções”.

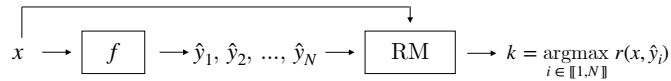
□ **PEFT** – O ajuste fino eficiente de parâmetros (*Parameter-Efficient FineTuning*, PEFT) é uma categoria de métodos utilizados para executar SFT de maneira eficiente. Em particular, a adaptação de baixo posto (*Low-Rank Adaptation*, LoRA) aproxima os pesos que podem ser aprendidos W fixando W_0 e aprendendo as matrizes de posto baixo A e B em seu lugar:

$$\begin{matrix} k \\ \downarrow \\ d \end{matrix} \begin{matrix} \boxed{W} \end{matrix} \approx \begin{matrix} k \\ \downarrow \\ d \end{matrix} \begin{matrix} \boxed{W_0} \end{matrix} + \begin{matrix} r \\ \downarrow \\ d \end{matrix} \begin{matrix} \boxed{B} \end{matrix} \times \begin{matrix} r \\ \downarrow \\ k \end{matrix} \begin{matrix} \boxed{A} \end{matrix}$$

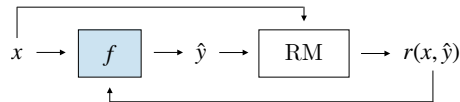
Observação: Outras técnicas de PEFT incluem ajuste de prefixos e inserção de camadas adaptadoras.

3.4 Ajuste de preferências

□ **Modelo de recompensas** – Um modelo de recompensas (*Reward Model*, RM) é um modelo que prediz quão bem uma saída $\hat{y}$ se alinha com o comportamento desejado dada a entrada x . A amostragem *Best-of-N* (BoN), também chamada de amostragem de rejeição, é um método que utiliza um modelo de recompensas para selecionar a melhor resposta entre N gerações.



□ **Aprendizado por Reforço** – O Aprendizado por Reforço (*Reinforcement Learning*, RL) é uma abordagem que utiliza o RM e atualiza o modelo f em função das recompensas recebidas por suas respostas geradas. Se o RM se basear em preferências humanas, esse processo é chamado de Aprendizado por Reforço a partir de Feedback Humano (*Reinforcement Learning from Human Feedback*, RLHF).

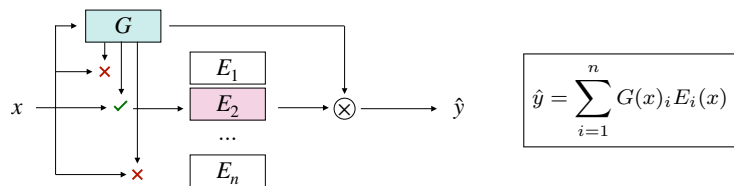


A Otimização de Políticas Próximas (*Proximal Policy Optimization*, PPO) é um algoritmo popular de RL que incentiva recompensas mais altas mantendo o modelo próximo do modelo base para prevenir o hackeamento de recompensas.

Observação: Também existem abordagens supervisionadas, como a otimização direta de preferências (Direct Preference Optimization, DPO), que combina RM e RL em uma única etapa supervisionada.

3.5 Otimizações

□ **Mistura de especialistas** – A mistura de especialistas (*Mixture of Experts*, MoE) é um modelo que ativa apenas uma porção dos seus neurônios no momento de inferência. Ele é baseado em uma porta G e especialistas $E_1, \dots, E_n$.



LLMs baseados em MoE utilizam esse mecanismo de porta em suas Redes Neurais Preliminadas (Feed-Forward Neural Networks, FFNNs).

Observação: Treinar um LLM baseado em MoE é notoriamente desafiador, como mencionado no artigo do LLaMA, cujos autores decidiram não utilizar essa arquitetura apesar de sua eficiência em tempo de inferência.

□ **Destilação** – *Destilação* é um processo em que um modelo estudante (pequeno) S é treinado utilizando as saídas previstas de um modelo professor (grande) T . O treinamento utiliza divergência KL como função de perda:

$$\text{KL}(\hat{y}_T || \hat{y}_S) = \sum_i \hat{y}_T^{(i)} \log \left(\frac{\hat{y}_T^{(i)}}{\hat{y}_S^{(i)}} \right)$$

Observação: Rótulos de treinamento são considerados rótulos “suaves”, uma vez que representam as probabilidades das classes.

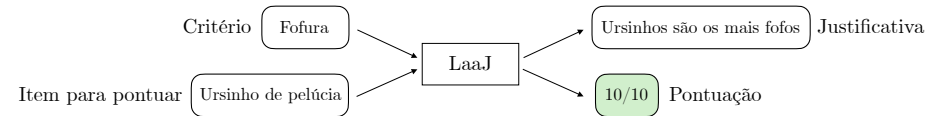
□ **Quantização** – *Quantização de modelo* é uma categoria de técnicas que reduzem a precisão dos pesos do modelo, limitando seu impacto no desempenho do modelo resultante. Como resultado, isso reduz o uso de memória pelo modelo e acelera a sua inferência.

Observação: QLoRA é uma variante quantizada do LoRA comumente utilizada.

4 Aplicações

4.1 LLM como Juiz

□ **Definição** – O LLM como juiz (*LLM-as-a-Judge*, LaaJ) é um método que utiliza um LLM para pontuar saídas de acordo com critérios definidos. Cabe destacar que o LaaJ também é capaz de gerar justificativas para a sua pontuação, o que auxilia a interpretabilidade.



Ao contrário de métricas anteriores aos LLMs, como ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*), o LaaJ não necessita de nenhum texto como referência, o que faz com que seja conveniente para avaliar qualquer tipo de tarefa. Em particular, o LaaJ apresenta forte correlação com avaliações humanas quando se baseia em um modelo grande e poderoso (por exemplo, GPT-4), pois exige capacidades de raciocínio para ter bom desempenho.

Observação: LaaJ é útil para realizar rodadas rápidas de avaliação, mas é importante monitorar o alinhamento entre as saídas do LaaJ e as avaliações humanas para garantir que não existam divergências.

□ **Vieses Comuns** – Modelos de LaaJ podem ter os seguintes vieses:

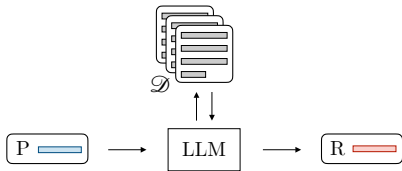
	Viés de posição	Viés de verbosidade	Viés de autorreforço
Problema	Favorece primeira posição em comparações por pares	Favorece conteúdo mais verboso	Favorece suas próprias saídas
Solução	Métrica média em posições aleatórias	Penalizar pelo tamanho da saída	Usar juiz de outro modelo base

Uma solução para esses problemas pode ser realizar um ajuste fino de um LaaJ customizado, mas isso requer bastante esforço.

Observação: A lista de vieses acima não é exaustiva.

4.2 RAG

Definição – A Geração Aumentada via Recuperação (*Retrieval-Augmented Generation*, RAG) é um método que permite ao LLM acessar conhecimento externo relevante para responder a uma dada pergunta. Isso é particularmente útil quando se deseja incorporar informações que ultrapassaram a data limite do conhecimento pré-treinado do LLM.



Dada uma base de conhecimento $\mathcal{D}$ e uma pergunta, o Recuperador obtém os documentos mais relevantes e Aumenta o prompt com as informações relevantes antes de Gerar a saída.

Observação: A etapa de recuperação normalmente depende de embeddings de modelos do tipo apenas codificador.

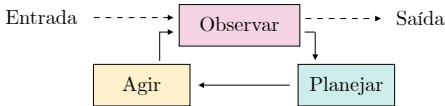
Hiperparâmetros – A base de conhecimento $\mathcal{D}$ é inicializada dividindo os documentos em fragmentos de tamanho n_c e convertendo-os em vetores de tamanho $\mathbb{R}^d$.



4.3 Agentes

Definição – Um agente é um sistema que persegue objetivos e completa tarefas de maneira autônoma em nome do usuário. Ele pode utilizar diferentes cadeias de chamadas de LLMs para isso.

ReAct – O ReAct, abreviação de Reason (raciocinar) + Act (agir), é um framework que permite múltiplas cadeias de chamadas de LLMs para completar tarefas complexas:



Esse framework é composto pelas seguintes etapas:

- Observar: Sintetiza ações prévias e expressa explicitamente o que se conhece até o momento.
- Planejar: Detalha as tarefas que devem ser realizadas e quais ferramentas utilizar.
- Agir: Realiza uma ação através de uma API ou busca informações relevantes na base de conhecimento.

Observação: Avaliar um sistema agente é desafiador. No entanto, é possível realizar avaliações tanto no nível de componente, através de entradas e saídas locais, quanto no nível de sistema, através de chamadas encadeadas.

4.4 Modelos de Raciocínio

Definição – Um modelo de raciocínio é um modelo que se baseia em cadeias de raciocínio do tipo CoT para resolver tarefas complexas em matemática, programação e lógica. Exemplos de modelos de raciocínio incluem a série o da OpenAI, DeepSeek-R1 e o Gemini Flash Thinking do Google.

Observação: O DeepSeek-R1 mostra explicitamente a sua cadeia de raciocínio entre as tags <think>.

Dimensionamento – Dois tipos de métodos de dimensionamento são utilizados para melhorar as capacidades de raciocínio:

	Descrição	Ilustração
Durante o treinamento	Executa RL por mais tempo para o modelo aprender a gerar raciocínio no estilo CoT antes de responder	Desempenho Etapas de RL
Durante o teste	Deixa o modelo pensar mais tempo antes de responder, usando palavras-chave como “Esperar”	Desempenho Tamanho do CoT